

1. Explain the difference between frontend, backend, and full-stack development with suitable real-world examples.

ANS- Frontend Development: The part of a website that the user directly sees and interacts with (design, buttons, layout). Technologies: HTML, CSS, JavaScript, React.

Example: Instagram's feed displaying, the like button changing color when clicked.

Backend Development: The server-side part that processes and stores data, not visible to the user. Technologies: Node.js, Python, MySQL.

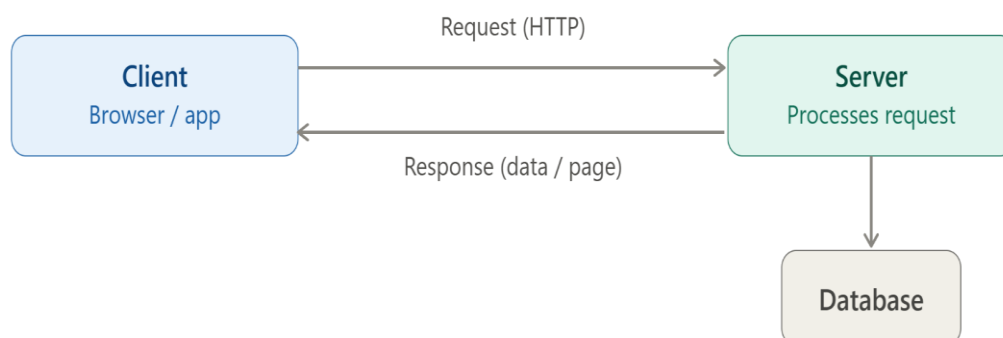
Example: When the like button is clicked, the count getting updated in the database.

Full-Stack Development: A combination of both frontend and backend — a developer who can handle both.

Example: An e-commerce site where the product page design (frontend) and the cart-saving logic (backend) are both built by the same developer.

2. Create a simple diagram showing how the client-server model works in web architecture.

Ans-



3. Describe how a browser requests and displays a web page from a web server.

Ans-Enter URL: User types the website address in the browser (e.g. www.example.com).

DNS lookup: Browser contacts a DNS server to convert the domain name into an IP address.

HTTP request: Browser sends an HTTP/HTTPS request to the web server at that IP address, asking for the page.

Server response: Server processes the request and sends back the requested files — HTML, CSS, JavaScript, images.

Page rendering: Browser reads these files and renders them on screen as the visible webpage.

4. Identify and list the tools required to set up a web development environment. Explain the purpose of each.

Ans-Code Editor / IDE (e.g. VS Code) — Used to write, edit, and organize code with features like syntax highlighting, extensions, and debugging support.

Web Browser (e.g. Chrome, Firefox) — Used to run and test the website, and includes Developer Tools to inspect HTML/CSS/JS and debug errors.

Version Control System (e.g. Git) — Tracks changes in code, allows reverting to previous versions, and enables collaboration between multiple developers.

Hosting Platform for Repositories (e.g. GitHub) — Stores project code online, enables team collaboration, and often integrates with deployment tools.

Runtime Environment (e.g. Node.js) — Allows JavaScript to run outside the browser, needed for backend development and running build tools/package managers.

Package Manager (e.g. npm) — Installs, manages, and updates libraries/dependencies required by the project.

Web Server / Local Server (e.g. XAMPP, Live Server extension) — Simulates a real server on the local machine to test how the website behaves before deployment.

Database System (e.g. MySQL, MongoDB) — Stores and manages data used by the application (user info, products, etc.).

API Testing Tool (e.g. Postman) — Used to test backend APIs by sending requests and checking responses without needing a frontend.

Browser Developer Tools (built into browsers) — Helps inspect elements, view console errors, check network requests, and debug frontend issues live.

5. Explain what a web server is and give examples of commonly used servers.

Ans- A web server is software (running on a computer) that **stores, processes, and delivers web pages** to clients (browsers) when they request them. It listens for incoming requests (usually over HTTP/HTTPS), processes them, and sends back the appropriate response — HTML pages, images, files, or data.

In short: it's the "waiter" that takes a client's request and serves back what was asked for.

Working:

1. Client sends a request (e.g. opening a URL).
2. Web server receives and processes the request.
3. Server sends back the requested content as a response.

Commonly used web servers

1. **Apache HTTP Server** — One of the oldest and most widely used, open-source, highly configurable.
2. **Nginx** — Known for high performance and handling many concurrent connections; also used as a reverse proxy/load balancer.

6. Define the roles of a frontend developer, backend developer, and database administrator in a project.

Ans-Frontend Developer

- Designs and builds the user interface (UI) that users interact with directly.
- Ensures the website is responsive, visually appealing, and user-friendly.
- Works with HTML, CSS, JavaScript, and frameworks like React.
- *Example:* Building the product listing page and "Add to Cart" button design on an e-commerce site.

Backend Developer

- Builds and manages the server-side logic that powers the application.
- Handles data processing, business logic, APIs, authentication, and security.
- Works with languages like Node.js, Python, Java and connects the app to the database.
- *Example:* Writing the logic that checks stock availability when "Add to Cart" is clicked.

Database Administrator (DBA)

- Manages, organizes, and secures the database where application data is stored.
- Ensures data integrity, performs backups, optimizes queries for performance, and controls access permissions.
- Works with database systems like MySQL, MongoDB, PostgreSQL.
- *Example:* Making sure the product and order data is stored efficiently and doesn't get corrupted or lost.

7. Install VS Code and configure it for HTML, CSS, and JavaScript development. Take a screenshot of the setup.

Ans-

Download VS Code: Visited the official website code.visualstudio.com and downloaded the installer suitable for the operating system (Windows/Mac/Linux).

Installation: Ran the downloaded installer, accepted the license agreement, and completed the setup using default settings. VS Code launched successfully after installation.

Installing extensions for web development: Opened the Extensions panel (Ctrl+Shift+X) inside VS Code and installed the following extensions:

- **Live Server** – runs the HTML file on a local development server and auto-refreshes the browser on save.
- **Prettier** – automatically formats HTML, CSS, and JavaScript code for consistency.
- **HTML CSS Support** – provides autocomplete suggestions for CSS class/id names inside HTML files.
- **Auto Rename Tag** – automatically renames the closing HTML tag when the opening tag is edited.

Creating a project folder: Created a new folder for the project and opened it in VS Code using File → Open Folder. Inside this folder, created three files: `index.html`, `style.css`, and `script.js`.

Linking files: Linked the CSS and JavaScript files to the HTML file using `<link>` and `<script>` tags respectively, so all three work together.

Testing the setup: Right-clicked on `index.html` and selected "Open with Live Server." This launched the page in the browser, and any changes made in VS Code reflected instantly on save — confirming the environment was correctly configured.

8. Explain the difference between static and dynamic websites. Provide an example of Each.

Ans-

Aspect	Static Website	Dynamic Website
Content	Fixed	Changes based on data/user
Technology	HTML, CSS	HTML, CSS, JS + backend + database
Speed	Faster	Slightly slower (processing involved)
Example	Portfolio site	E-commerce site (Amazon)

9. Research and list five web browsers. Explain how rendering engines differ between them.

Ans- A **rendering engine** is the part of a browser that takes HTML, CSS, and JavaScript and converts it into the visual webpage you see on screen. Different engines can interpret code slightly differently, causing pages to look or behave differently across browsers.

1. **Blink** (Chrome, Edge, Opera)

- Developed by Google, forked from WebKit in 2013.
- Fast performance, strong support for the latest web standards and JavaScript features.
- Since most browsers now use Blink, it has become the "default" behavior most developers test against.

2. **Gecko** (Firefox)

- Developed by Mozilla, open-source.
- Known for strict adherence to web standards and strong privacy/security features.
- Sometimes renders CSS spacing, fonts, or animations slightly differently than Blink

3. WebKit (Safari)

- Developed originally by Apple (also based on the same source that Blink forked from).
- Optimized for Apple devices — better battery efficiency and smooth performance on Mac/iOS.
- Slower to adopt certain new web standards compared to Blink, causing occasional compatibility issues for developers.

10. Draw a labeled diagram showing the basic web architecture flow — client, server, database, and APIs.

Ans- **Client:** The user-facing part (browser or app) where requests originate.

- **API:** Acts as a middleman — defines rules for how the client and server communicate (what data can be requested, in what format).
- **Server:** Receives the request via the API, processes the business logic, and interacts with the database if data is needed.
- **Database:** Stores and retrieves the actual data (users, products, etc.) requested by the server.

Example flow: Client clicks "View Profile" → API sends this request to the server → server queries the database for user data → database returns the data → server sends it back through the API → client displays the profile on screen

